

LISTA DE CHEQUEO AMPLIADA PARA SEGUIMIENTO DE PROYECTOS DE SOFTWARE

Categoría	Ítem a revisar	Estado (OK / X / %)	Observaciones
1. Requisitos y planificación	Objetivo del proyecto definido claramente		
	Historias de usuario o casos de uso redactados		
	Backlog de funcionalidades priorizadas		
	Metodología ágil utilizada (Scrum, Kanban)		
	Planificación de sprints o tareas		
2. Arquitectura del software	Uso de una arquitectura clara (MVC, 3 capas, hexagonal, etc.)		
	Separación clara entre capas (modelo, lógica, presentación)		
	Backend desacoplado del frontend mediante API REST		
	Uso de controladores, servicios y repositorios		
	Aplicación de principios SOLID en la estructura del código		
3. Patrones de diseño	Aplicación de al menos un patrón de diseño (Factory, Singleton, Repository, etc.) donde sea pertinente		
	Patrón DTO (Data Transfer Object) para respuestas		
	Uso de middleware/interceptores para validación o logging		
4. Buenas prácticas en el backend	Formato uniforme de respuesta de API (status, message, data, error)		
	Manejo centralizado de errores (try/catch + middleware o interceptores)		
	Validación de entradas (DTOs, schemas, Joi, class-validator, etc.)		



Categoría	Ítem a revisar	Estado (OK / X / %)	Observaciones
	Uso correcto de códigos HTTP (200, 201, 400, 404, 500, etc.)		
	Modularidad del código y reutilización de funciones		
	Separación de lógica de negocio del acceso a datos		
5. Base de datos	Modelo ER diseñado y actualizado		
	Uso de llaves primarias y foráneas correctamente		
	Uso de ORMs (Sequelize, TypeORM, Prisma) o SQL estructurado limpio		
	Migraciones o scripts para creación de base de datos		
	Relación clara entre entidades y sus modelos		
6. Frontend	Arquitectura de componentes o páginas clara		
	Consumo eficiente de la API (fetch, axios, async/await)		
	Manejo de estado (useState, Redux, Pinia, etc.) según el framework		
	Validación de formularios en el cliente		
	Mensajes claros de error y éxito al usuario		
	Responsividad (uso de media queries, flexbox/grid, frameworks como Bootstrap o Tailwind)		
7. Estándares de respuesta (backend)	Todas las respuestas usan un formato estándar como: { status: 'success' }	'error', message: 'texto', data: {}, error: {} }	
	Los errores manejan trazabilidad (logging, timestamp, stacktrace en desarrollo)		
	Se devuelve solo lo necesario (evitar sobrecargar la respuesta)		
8. Pruebas	Pruebas unitarias en backend (Jest, Mocha, etc.)		

Categoría	Ítem a revisar	Estado (OK / X / %)	Observaciones
	Pruebas manuales documentadas con escenarios		
	Flujo completo funcional de extremo a extremo (frontend-backend-db)		
9. Herramientas y DevOps	Uso de Git con ramas (main, dev, feature/*)		
	Commits significativos y frecuentes		
	CI/CD básico (GitHub Actions, Vercel, Netlify, etc.)		
	Entornos separados para desarrollo y producción		
	Uso de variables de entorno seguras (.env)		
10. Documentación	README con: – Descripción del proyecto – Instrucciones de instalación – Stack tecnológico – Cómo correr backend y frontend		
	Documentación de la API (Swagger, Postman, Redoc)		
	Diagrama de arquitectura (opcional)		
	Guía para contribuir o entender el código		
11. Despliegue	Backend desplegado correctamente (Railway, Render, etc.)		
	Frontend desplegado correctamente (Vercel, Netlify, etc.)		
	API y frontend funcionan en conjunto en producción		
	Base de datos alojada en entorno accesible (ElephantSQL, PlanetScale, Supabase, etc.)		
12. Reflexión y aprendizaje	Retroalimentación recibida e integrada		
	Reflexión de cada integrante sobre su rol y aprendizajes		

Formato uniforme de respuesta en backend (ejemplo JSON)



```
{
  "status": "success",
  "message": "Usuario creado correctamente",
  "data": {
    "id": 1,
    "nombre": "Carlos Pérez"
  },
  "error": null
}
{
  "status": "error",
  "message": "El correo ya está en uso",
  "data": null,
  "error": {
    "code": "DUPLICATE_EMAIL",
    "details": "El campo 'email' debe ser único"
  }
}
```

Herramientas sugeridas

Área	Herramientas sugeridas
Repositorio de código	Git + GitHub/GitLab
API REST	Node.js + Express / NestJS / Django REST / FastAPI
Base de datos	PostgreSQL / MySQL / MongoDB + ORM
Frontend	React / Vue / Angular
Pruebas	Jest, Mocha, Cypress
Documentación API	Swagger / Postman / Redoc
Despliegue	Vercel, Netlify, Render, Railway, Supabase
CI/CD	GitHub Actions, GitLab CI
Gestión de tareas	Trello, Jira, Notion, GitHub Projects

